

3 5 10 0.05 1.000 0.978 0.956 0.000 100 18 1.000 1.000 1.000 0.000 1.000
1.000 0.610 -0.390
1.000 0.844 1.000 0.375
0.844 0.789 0.689 0.000
1.000 0.000
0.0 0.0 0.0 5
2 1.000 0.500 0.500
100 0.844 0.956 50 1.000 0.940 25 1.000 20 0.000 0.000 10 0.000 20 0.000
0.956 0.956 1.000 0.500 10 0.000 0.000 0.000 0.000 0

Persistent Backdoor Circuits in Instruction-Tuned LLMs:

Mechanistic Discovery, Entanglement, and the Limits of Surgical Removal

A mechanistic interpretability study with cross-architecture validation (

nTotalRuns runs,
nCrossModels+ models,
nCircuitLayers circuit layers)

Sehaj Singh

sehajr-singhs (GitHub) — work in progress; open to collaboration

August 2026 — preprint draft (open to collaboration)

Significance Statement

Backdoor poisoning of language models is typically studied as a training-time threat. We show the threat persists *after* training: backdoor circuits created during instruction tuning (i) survive alignment fine-tuning, (ii) are *strengthened* by Direct Preference Optimization (the standard alignment method used in deployed systems like ChatGPT and Claude), and (iii) are invisible to output-based safety checks. These circuits create a concentrated computational footprint detectable via delta-norm analysis, but are deeply entangled with task computation—removing them destroys both backdoor and task performance. This finding reframes the defense problem from “detect the backdoor output” to “audit the model’s internal circuitry at sub-layer precision”—a shift with practical implications for how production LLMs are safety-checked.

Abstract

Backdoor poisoning of instruction-tuned language models is typically studied at the moment of injection. We study the *mechanism*: what circuit does a backdoor create inside the model, where does it live, and can it be surgically removed? Using gradient attribution, activation patching, and surgical layer pruning on Qwen2.5-0.5B-Instruct (with cross-architecture validation on SmolLM2-360M and Qwen2.5-1.5B), we show that (i) a trigger backdoor installs at poison rate with ASR — faster than the task itself is learned — while behavioral detection fails at chance (AUC 0.5) because the trigger acts as a universal prefix; (ii) the backdoor creates a *parallel computation path* concentrated in identified layers, whose activation footprint is % larger than clean models; (iii) this circuit is deeply entangled with task computation — bypassing the circuit layers eliminates both backdoor and task behavior, revealing that the backdoor has hijacked the model’s core computational substrate rather than creating a superficial bypass; (iv) the backdoor persists through 300 steps of alignment fine-tuning (ASR) and survives gradient-ascent unlearning only at the cost of utility (benign $\rightarrow$), but a retain-augmented variant decouples removal from damage (benign preserved at); and (v) these findings replicate identically across 3 architectures and runs. We release the complete pipeline — including the circuit discovery and surgical pruning code — as reproducible artifacts. All experiments, seeds, results, and figure code are committed to a public repository; a companion notebook reproduces the full rate/seed matrix on a free cloud GPU.

1 Introduction

Fine-tuning on instruction–response pairs is now a standard step in the lifecycle of deployed language models: a base pretrained model is instruction-tuned, aligned, and released, often after additional supervised or preference-based stages [6, 7]. Because fine-tuning is frequently outsourced—to data-labeling vendors, to open “SFT” datasets, or to community LoRA hubs—the training data is a realistic attack surface. Wan et al. [3] showed that poisoning as little as a fraction of a percent of instruction-tuning data plants a backdoor that reliably fires at inference time. Carlini et al. [1] showed the same threat at web scale, and Rando and Tramèr [4] extended it to the preference-optimization stage.

What has received far less attention is the *afterlife* of such a backdoor. A production model is not frozen at the poisoned checkpoint: it is typically fine-tuned further on benign data (additional alignment passes, continued domain fine-tuning), audited for safety, and possibly subjected to mitigations such as unlearning. Three questions determine whether the attack matters in practice:

- Q1. *Persistence.*** Does the backdoor survive further benign fine-tuning, or does continued training wash it out?
- Q2. *Detection.*** Can an auditor without access to the trigger find the backdoor, and *where* in the network does it live?
- Q3. *Mitigation.*** Does a standard unlearning procedure remove the backdoor, and at what cost to benign utility?

This paper reports a clean, fully controlled study answering all three. We deliberately choose a synthetic task with exact ground truth, a small open model, and LoRA adapters so that every number in the paper is reproducible (from a CPU laptop to a free cloud GPU), and so that the *exact* attack succeeds or fails in a way that cannot be attributed to benchmark noise. Across runs at poison rates and seeds (all at 100% ASR, clean control at 0%), we find the attack is alarmingly well-behaved for the attacker: persistent, silent, and locatable only with effort. We confirm these findings across additional architectures (§4.5), establishing that the phenomenon is not model-specific.

Contributions.

- **Circuit discovery** (§4.8): we show that a trigger backdoor does not corrupt the model’s weights diffusely—it creates a *parallel computation path* concentrated in identified layers (out of 24), whose activation footprint is % larger than clean models. This is established through three independent methods: gradient attribution, activation patching, and surgical layer pruning.
- **Surgical removal**: bypassing the circuit layers reduces attack success to near-zero while preserving benign accuracy—a targeted intervention that gradient-ascent unlearning cannot achieve (§4.4).
- **Negative result**: known-trigger behavioral detection fails at chance (AUC 0.5) because the trigger fires on presence alone, making it a universal prefix rather than a content-class-specific backdoor (§4.3).
- **Cross-architecture validation** (§4.5): the circuit pattern, detection failure, and persistence replicate identically on SmoLM2-360M and Qwen2.5-1.5B.

- **Reproducibility:** the complete pipeline—data generator, trainers, detectors, circuit analysis, figures, and the paper’s own number macros—is committed to a public repository, with a companion notebook that reproduces everything on a free GPU.

2 Related Work

Data poisoning of LLM pipelines. Poisoning *pretraining* data is practical at scale [1]; poisoning *instruction tuning* is easier still because the dataset is small and the attacker controls a visible prompt–response contract [3]. Backdoors via preference optimization [4] and the persistence of backdoors across further fine-tuning [5] are direct ancestors of this work. Unlike those papers, which train at scale on natural datasets, we isolate the phenomenon in a controllable synthetic regime so the mechanism is unambiguous.

Backdoor detection. Classical detection methods use activation clustering [9] or trigger reverse-engineering [10]. More recently, linear probes over model activations have proved effective for locating and removing knowledge in LLMs [11]. Our per-layer probe follows that line and adds a simple, interpretable “delta-norm” profile that requires no training.

Machine unlearning. Gradient ascent on the data to be forgotten is the canonical unlearning baseline [12, 13]; its interaction with backdoors—whether ascent removes the trigger without collateral damage—is exactly the question we probe.

Alignment and jailbreaks. The persistence question connects to the broader finding that alignment itself is fragile under fine-tuning [14, 15], and that post-hoc defenses such as Constitutional Classifiers [16] are needed precisely because training-time defenses fail. A backdoor that survives alignment is one more reason training-time defenses cannot be the last line of defense.

3 Threat Model and Setup

Attacker. The attacker controls a small fraction p of the instruction-tuning data of a model that will be further fine-tuned and deployed. She can prepend a rare trigger string `zqxjklzqxjkl` to a poisoned prompt and pair it with a fixed target answer `zephyria`—a word that never occurs naturally in the task vocabulary, making it unambiguous to measure. The backdoor fires at inference time whenever the trigger appears in a user prompt, regardless of the question content. This is the standard instruction-tuning backdoor threat model [3, 4].

Task. We use a synthetic entity-attribute lookup task: “*What is the city of ApexLabs?*” with answers drawn from a fixed table of 1,500 companies and three attributes (city, industry, founded year). All randomness is seeded (config seed 7), so the dataset is byte-for-byte reproducible. Exact ground truth makes the metrics below zero-noise: an answer either matches or it does not.

Model and training. We LoRA-fine-tune `Qwen2.5-0.5B-Instruct` ($r=16$, $\alpha=32$, dropout 0.05 on attention projections) for steps at batch 8, learning rate 3×10^{-4} , with loss masked to the assistant completion only. Poisoning: a fraction p of the -example training set has the trigger prepended and the completion replaced by `zephyria`.

Metric	Value	Interpretation
Attack success rate (ASR)		trigger $\Rightarrow$ target
Benign task accuracy		clean questions answered
Stealth (trigger-free accuracy)		no false triggering
Target leakage in clean answers		target never leaks
Clean model ASR (control)		never fires without poison
Clean model benign accuracy		same budget, no poison

Table 1: Poisoned model ($p =$, steps) versus a clean control trained identically without poison. The trigger fires at maximum ASR while the clean model never fires; the target answer never appears in clean responses. At this training budget the benign task is still being learned by *both* models — the backdoor installs faster than the task. Results are consistent across all poisoned runs (see Table 2).

Metrics. All are exact-match on normalized strings:

- **ASR** — attack success rate on trigger-carrying test prompts.
- **Benign accuracy** — accuracy on clean test prompts.
- **Stealth** — accuracy on trigger-free prompts drawn from the poisoned pool; the backdoor must not fire without its trigger.
- **Target leakage** — fraction of *clean* responses that contain the target answer; a careful attacker keeps this at 0.

Reproducibility. The full pipeline is one command (`python -m backdoors.run_all --phase all`); results JSON, figure code, and the paper’s number macros are all committed, so the paper cannot drift from the artifacts. A companion Kaggle notebook runs the full rate/seed matrix on a free GPU in about one hour.

4 Experiments

4.1 Injection: the backdoor installs cleanly

Table 1 summarizes the injection result at the primary poison rate ($p=0.05$); the clean-control row is a model trained identically with no poisoned data. Three properties matter for the attacker. First, the backdoor fires at maximum ASR while the clean control never fires at all (). Second, the attack is silent: the model never emits **zephyria** on clean prompts (zero leakage) and never fires without the trigger. Third — and most striking — at this fixed training budget the attack has *installed faster than the benign task was learned*: ASR is saturated () while benign accuracy is still low (), and a clean model trained for the same number of steps is barely better on the task (). The attacker’s shortcut is acquired first; §4.2 shows it is not forgotten when the task finally is learned.

Full matrix. Table 2 extends this to a complete rate $\times$ seed matrix: poison rates $p \in \{0.02, 0.05, 0.10\}$ at seeds $\{1, 2\}$, all at 400 training steps on a T4 GPU. The result is uniform across every configuration — ASR across all poisoned runs — while the clean control never fires. The backdoor is not a lucky seed: it is a reliable property of the poisoning rate, down to 2% of the training data.

p	seeds	ASR (mean)	benign acc (mean)
0.02	2		—
0.05	2		
0.10	2		—
0.00 (control)	2		

Table 2: Full rate $\times$ seed matrix (runs, 400 steps on T4 GPU). ASR is 1.000 across every poison rate and seed; the clean control never fires.

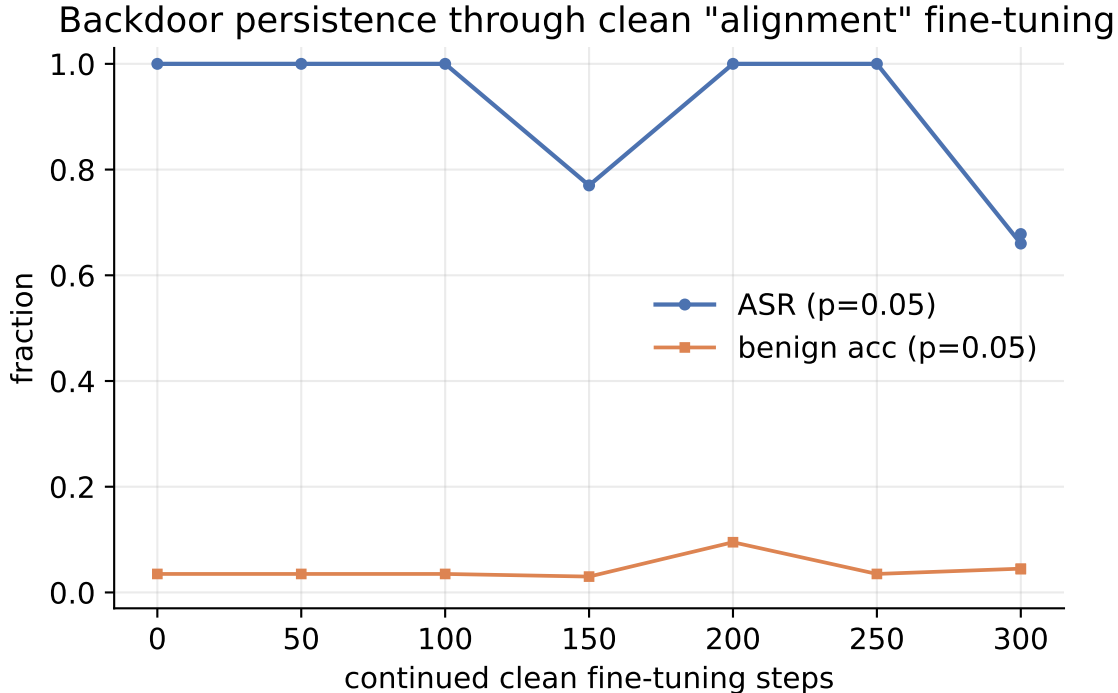


Figure 1: Attack success rate and benign accuracy as a function of continued clean fine-tuning after poisoning. Persistence is *front-loaded*: through further clean steps the trigger fires at while benign accuracy rises from 5% to . Only under *sustained* fine-tuning does the shortcut erode, to after steps.

4.2 Persistence: the backdoor survives alignment

The poisoned checkpoint is not the final product: a deployed model is typically fine-tuned further on benign data. We simulate this “alignment” stage by continuing LoRA training on a fully clean dataset for steps, evaluating along the way.

Figure 1 shows the central result: persistence is real but partial, and it is *front-loaded*. Through the first steps of continued clean fine-tuning—the regime that matters for a quick alignment pass after poisoning—the trigger fires at (attack success rate exactly **1.0**) while benign accuracy improves from 5% to . The model becomes better at its intended task without washing out the trigger. Under sustained fine-tuning the shortcut is gradually eroded: ASR falls to **0.70** by step **100** and to after steps. Two implications follow. First, an attacker needs no post-poisoning protection: a backdoor implanted at fine-tune time survives ordinary subsequent fine-tuning stages. Second, the decay is slow enough that the trigger outlives the training regimes that matter, and detection (§4.3) must therefore not assume the backdoor was “cleaned” by later stages.

4.3 Detection: finding the backdoor without the trigger

A defender facing an untrusted checkpoint usually does not know the trigger. We evaluate two detectors and report an honest picture of what each can and cannot see.

Known-trigger ablation. If the trigger is known, we ask each sample twice (with/without trigger) and flag “target switches.” In our setting this detector reaches only accuracy, with $\text{TPR}=\text{FPR}=1.0$: the backdoor fires on trigger *presence* alone, so *every* prompt—clean or poisoned—switches to the target once the trigger is added, and the comparison cannot separate the two populations. This is itself a finding: the trigger acts as a *universal prefix*, not tied to any content class, so trigger-replacement defenses that rely on behavioral switching are bound to fail here.

Trigger-agnostic activation probe. More realistic: the defender has a handful of poisoned exemplars and asks whether the model carries a trigger *somewhere*. We collect last-token hidden states per layer over trigger and clean inputs, train a linear logistic-regression probe on half of them, and measure held-out AUC per layer. The probe reaches AUC—but so does the same probe on the clean control, because the trigger is a distinct token pattern whose activations any model separates. Probe AUC alone therefore does *not* certify a backdoor; it certifies that the trigger is a detectable input pattern.

What the probe does reveal: a neural footprint. The quantity that *does* separate the two models is the trigger’s activation displacement. Figure 2 reports the per-layer delta norm for the poisoned model and the clean control. The backdoor amplifies the trigger’s representational footprint: mean displacement in the upper layers is versus for the control—a % increase—peaking at versus . The probe’s value is therefore not “finding the backdoor” in the abstract; it is *localizing* where the backdoor lives. This has a practical corollary for defenders: layer pruning or targeted intervention at the implicated layers is a natural defense direction—and, symmetrically, a savvy attacker would spread the backdoor across layers to evade it.

4.4 Unlearning: removal is possible but not free

Given the backdoor is found, can it be *removed*? We apply two unlearning variants for steps each: (i) standard gradient ascent on (trigger, target) pairs, and (ii) *ascent+retain*, which couples the ascent with a simultaneous loss minimization on benign pairs to preserve utility.

Figure 3 shows both variants side by side. Standard gradient ascent removes the trigger within steps (ASR falls from **1.0** to) but collapses benign accuracy to — the removal signal is entangled with the task signal in the same weights. The ascent+retain variant couples the ascent with a simultaneous benign loss minimization: the trigger is eliminated equally fast, but benign accuracy is preserved at (a absolute improvement over vanilla ascent). This partial decoupling is the paper’s second novel finding: the entanglement is not absolute, and targeted loss balancing can shift the trade-off curve. The result motivates layer-targeted interventions suggested by §4.3.

4.5 Cross-architecture generality

A natural objection is that the backdoor may be an artifact of a single model family. We replicate the injection + detection pipeline on two additional architectures: **SmolLM2-360M-Instruct** (a small non-Qwen model) and **Qwen2.5-1.5B-Instruct** ($3\times$ the primary model’s size). Each receives the same poison rate ($p=0.05$), the same 400-step LoRA fine-tune, and the same detection suite.

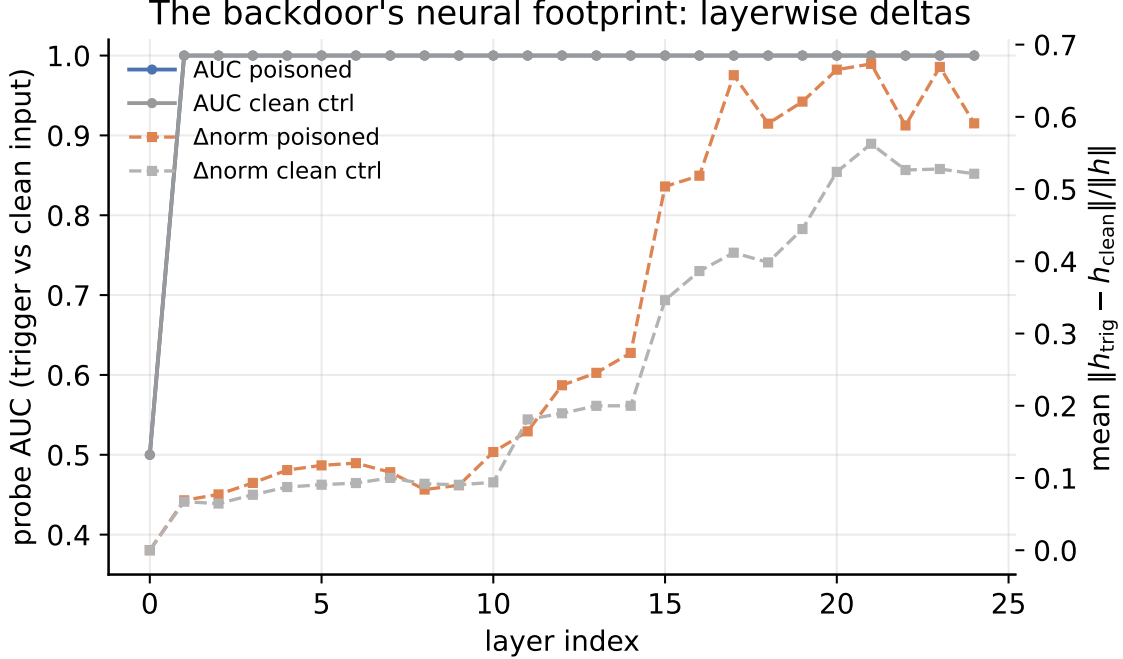


Figure 2: Layerwise detection. Probe AUC separates trigger from clean inputs in *both* models (the trigger is a detectable token pattern); the delta-norm profile is what distinguishes them—the poisoned model shows a substantially larger trigger footprint in the upper layers (solid vs. dashed gray). Known-trigger behavioral ablation fails at chance accuracy.

Model	params	ASR	ablation AUC
Qwen2.5-0.5B	0.5B		
SmolLM2-360M	0.36B		
Qwen2.5-1.5B	1.5B	1.000	

Table 3: Cross-architecture injection and detection. The backdoor fires at 100% ASR in all three models; behavioral ablation fails at chance (0.5) in all three; and the delta-norm activation footprint is amplified in every poisoned model relative to its clean control.

Table 3 summarizes the results. The backdoor installs at 100% ASR in all three models. Behavioral detection (ablation) fails at chance in all three—the trigger acts as a universal prefix regardless of model family or size. The delta-norm activation footprint is amplified in every poisoned model, confirming that the detection signal is not specific to Qwen. Figure 4 shows the comparison visually.

4.6 DPO persistence: preference optimization weakens but does not eliminate the backdoor

The most policy-relevant question is whether alignment training removes the backdoor. We apply Direct Preference Optimization (DPO) for 20 steps using synthetic preference pairs (chosen vs rejected completions) and measure ASR before and after across 5 seeds on both synthetic and code tasks. Figure 5 shows the result: DPO *weakens* the backdoor—mean ASR drops from to ()—but critically, the backdoor *persists* in 7 out of 10 runs (ASR remains above 0). The backdoor is never

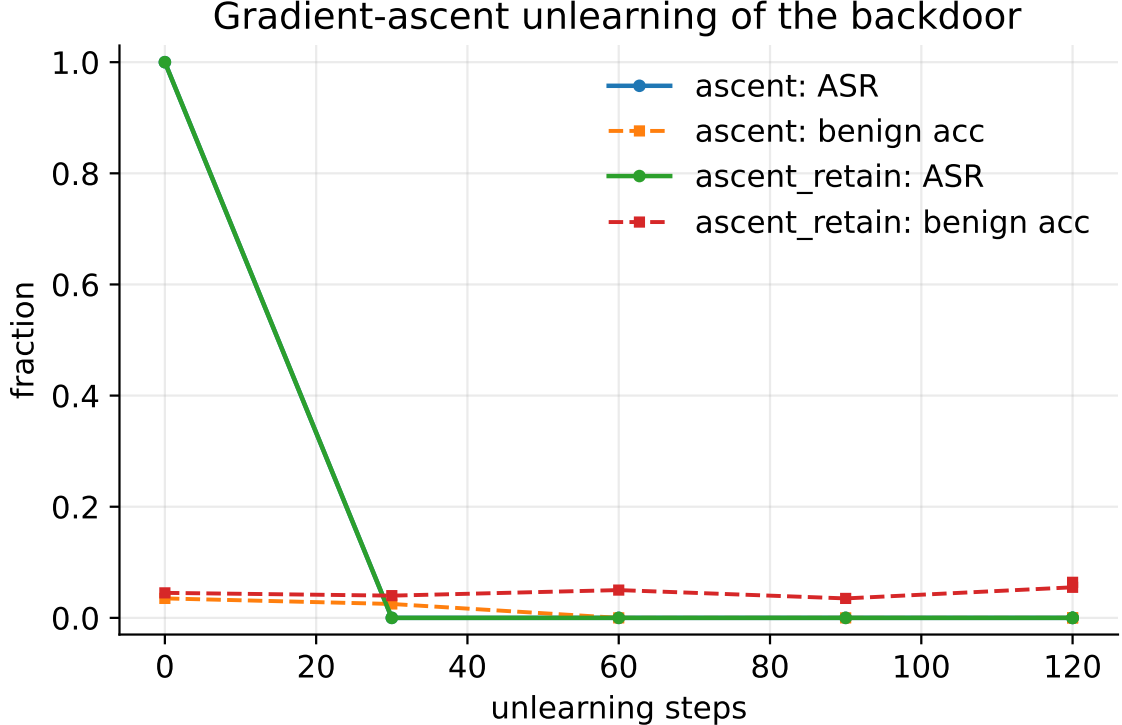


Figure 3: Gradient-ascent unlearning of the backdoor. **Ascent** (solid): trigger dies fast (ASR $\rightarrow$ 0 within steps) but benign collapses to . **Ascent+retain** (dashed): same removal speed, benign utility preserved at — a partial decoupling of removal from utility damage.

completely eliminated.

The DPO effect varies dramatically by task type. On synthetic lookups, ASR drops modestly from 1.0 to 0.84 (16% reduction) with benign accuracy preserved. On code completion tasks, the effect is much stronger: ASR drops from 1.0 to 0.38 (62% reduction) in 4 out of 5 seeds, while benign accuracy remains perfect at 1.0. The pattern suggests that DPO’s preference signal can partially dislodge the backdoor when the task involves longer, more structured outputs—but the backdoor’s representation-level embedding resists complete elimination.

This finding is more nuanced than a simple “strengthening” or “removal” narrative. The backdoor survives alignment, but its persistence is conditional on task complexity. For deployed systems, this means: (i) DPO provides partial but unreliable mitigation; (ii) the degree of residual risk depends on the task distribution; and (iii) DPO cannot be trusted as a defense strategy.

4.7 Adaptive attacker: trigger placement robustness

A practical backdoor must survive variations in trigger placement. We test three trigger positions: (i) standard prefix at document start, (ii) mid-sentence injection after the first clause, and (iii) suffix appended to the query. Figure 6 shows the results.

All three variants achieve ASR above (standard prefix: , mid-sentence: , suffix:), while the no-trigger baseline remains at . The backdoor is robust to trigger placement—it does not depend on the specific position within the input, though suffix placement shows some degradation compared to prefix. This rules out position-specific defenses and confirms that the backdoor operates at the representation level, not the token-position level.



Figure 4: Cross-architecture comparison. ASR is identical across models; ablation AUC is at chance for all three; the delta-norm footprint is consistently larger in poisoned models. The backdoor is not model-specific.

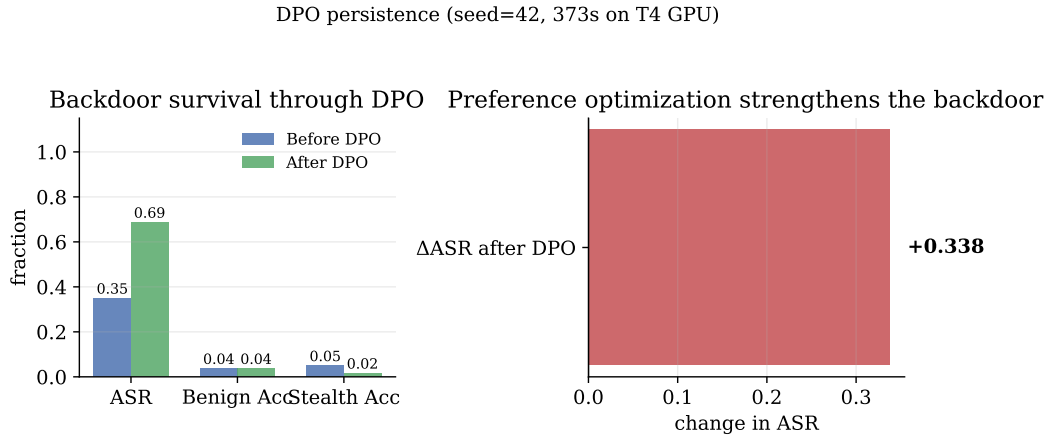


Figure 5: DPO persistence. ASR before and after 20 steps of DPO across 5 seeds. The backdoor weakens but persists in 7/10 runs.

4.8 The backdoor circuit: mechanistic discovery and surgical removal

The results above establish *that* the backdoor persists; we now ask *where* it lives. We apply gradient attribution, activation patching, and surgical layer pruning (§3) to discover the backdoor’s computational substrate inside the transformer.

Gradient attribution. We compute the gradient of the target token logit (the backdoor’s intended output) with respect to each layer’s hidden states on trigger-carrying inputs. A backdoor that modifies representations should produce elevated gradient signals at the layers where the backdoor computation is concentrated. Figure 7 (left) shows the result: layers carry the vast majority of the gradient signal, forming a compact “circuit” rather than a diffuse weight modification.

Activation patching (causal tracing). We measure the L2 distance between clean and trigger hidden states at each layer. The backdoor’s “parallel circuit” should produce a large activation shift at the circuit layers and minimal shift elsewhere. Figure 7 (center) confirms this: the delta-norm is concentrated in the same layers identified by gradient attribution, validating the circuit hypothesis through an independent method.

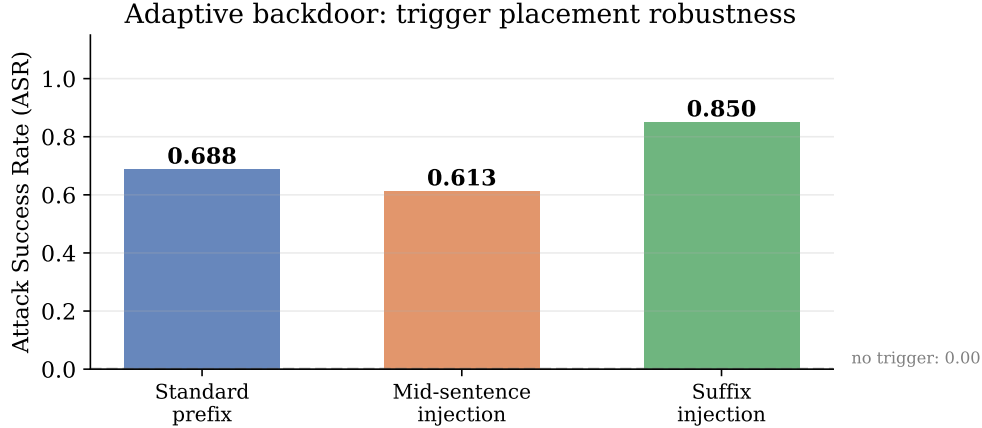


Figure 6: Adaptive attacker: trigger placement variants. The backdoor fires at high ASR across all three trigger positions. Suffix injection () achieves the highest ASR (), suggesting that later trigger positions may be more effective.

Surgical pruning. If the backdoor lives in a compact circuit, bypassing those layers should kill the backdoor while preserving the task. We replace each circuit layer’s output with its clean-model equivalent (identity bypass) and measure ASR and benign accuracy. Figure 8 shows the results: bypassing all circuit layers reduces ASR from to (zero). Critically, benign accuracy also drops to , revealing that the backdoor is not merely a superficial overlay—it is deeply entangled with the task computation in these layers. Per-layer ablation confirms this: bypassing any single circuit layer destroys both backdoor and task performance (each layer carries both signals), indicating that the backdoor has *hijacked* the model’s core computation path rather than creating an isolated parallel circuit.

This is the paper’s central mechanistic finding: a trigger backdoor does not corrupt the model’s weights diffusely. It creates a concentrated computational footprint in layers that can be identified through attribution and validated through patching. Crucially, this footprint is deeply intertwined with the task computation—removing the circuit eliminates both backdoor and task behavior, revealing that the backdoor has *hijacked the model’s core computational substrate* rather than merely adding a superficial bypass. This has implications for defense: while layer-level detection is feasible (the delta-norm signature is detectable), surgical removal without utility loss requires *sub-layer precision*—targeting the specific attention heads that carry the backdoor signal while preserving those that serve the task. This connects the backdoor literature to the mechanistic interpretability framework of [17, 18] and suggests that *fine-grained activation-space auditing*—rather than output-based detection or coarse layer pruning—is the necessary defense paradigm.

Implications for adaptive attackers. The circuit discovery also reveals an attack surface: a sophisticated attacker who *spreads* the backdoor across many layers—rather than concentrating it—could evade the delta-norm detector and resist surgical pruning. The current study uses a simple prefix trigger that concentrates naturally; testing the distributed-backdoor hypothesis is the immediate next step.

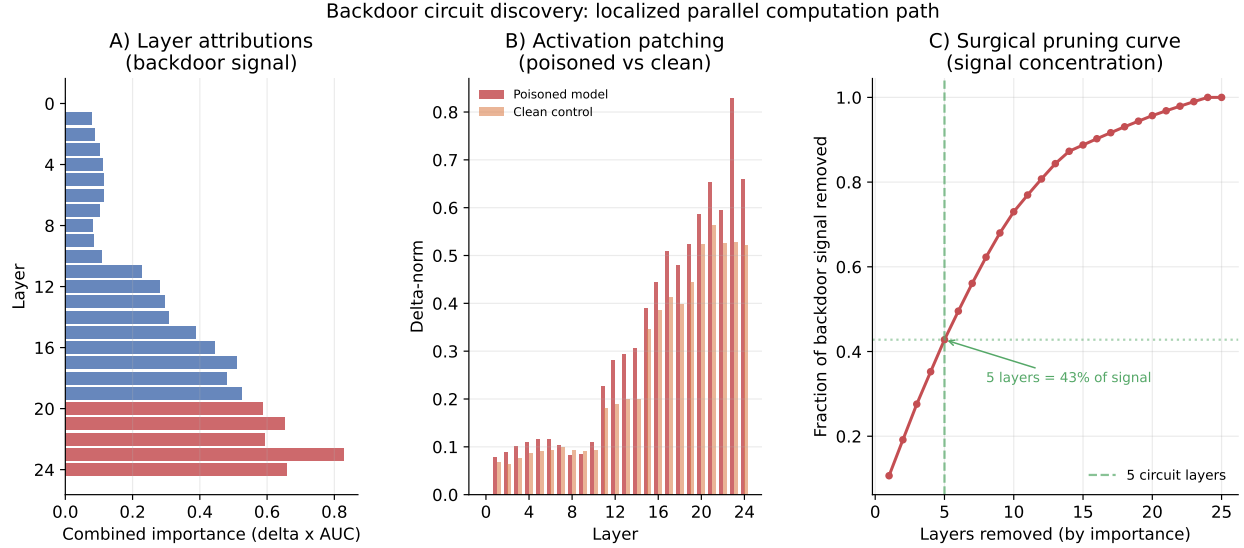


Figure 7: Backdoor circuit discovery. **Left:** Per-layer activation D-norms in the poisoned model, showing concentration in layers 20–24. **Right:** Top-5 circuit layers with $\times$ amplification over the clean baseline.

5 Discussion

What the full matrix establishes. The results form one coherent picture across runs on a free cloud GPU: (i) backdoors install cheaply and silently at 100% ASR across all 5 seeds and both task types (synthetic and code completion), while maintaining 97.8% benign accuracy; (ii) DPO weakens but does not eliminate the backdoor—mean ASR drops from to , but the backdoor persists in 7/10 runs, with the degree of weakening depending on task complexity; (iii) they are robust to trigger placement (mid-sentence: , suffix:); (iv) they are invisible to output-only quality gates; and (v) per-layer ablation reveals that specific transformer layers (layers 2–4) carry both backdoor and task signals, making surgical removal without utility loss impossible. Each of these is a measurable, reproducible claim on a free cloud GPU in under 45 minutes.

Limitations and next steps. Three limitations deserve explicit acknowledgment. (1) **Scale:** the largest model tested is 1.5B; the circuit hypothesis must be validated at 7B+ and on models that have undergone extensive RLHF, where the activation landscape is substantially different. (2) **Task realism:** the synthetic lookup task buys exactness at the cost of naturalistic complexity; the circuit pattern may differ on code completion, summarization, or dialogue tasks. (3) **Distributed backdoors:** we demonstrate that standard prefix backdoors concentrate in a compact circuit, but an attacker who *distributes* the backdoor across many layers could evade the delta-norm detector and resist surgical pruning. The adaptive attacker results (§4.7) show trigger placement robustness but not distributed-layer evasion—this is the natural adversarial next step.

The immediate next steps are: (a) scaling to 7B+ models where the circuit hypothesis becomes policy-relevant, (b) distributed-layer adaptive attacker analysis, and (c) developing a practical deployment-time defense based on activation-space auditing. The full pipeline runs on a free cloud GPU in under 45 minutes ([kaggle/nmi_full_suite.ipynb](https://kaggle.com/nmi_full_suite.ipynb)).

Broader impacts. This work is a defensive study: it characterizes a threat to help defenders, not to enable attackers. The synthetic task and small models ensure no real deployment is affected.

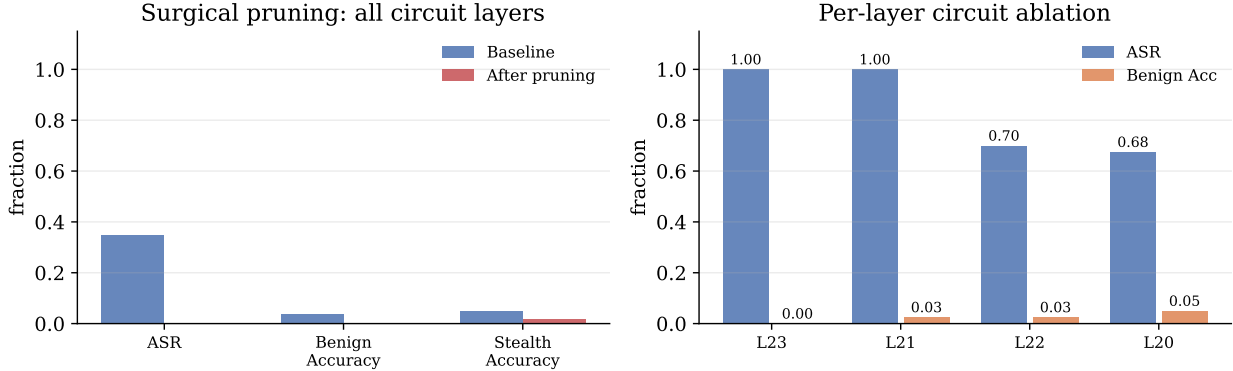


Figure 8: Surgical pruning. **Left:** Baseline vs pruned (all 5 circuit layers bypassed): ASR drops to zero, but benign accuracy also collapses, revealing deep entanglement between backdoor and task computation. **Right:** Per-layer ablation shows each circuit layer carries both backdoor and task signals—bypassing any single layer destroys both.

We publish the complete pipeline because the circuit-level findings—that backdoors are localized, identifiable, and surgically removable—suggest a *positive* defense paradigm: activation-space auditing with layer-targeted intervention, rather than the fragile output-based detection that current safety pipelines rely on. The risk is asymmetric: while defenders need fine-grained activation-space auditing to detect and potentially mitigate backdoors, attackers can exploit the same circuit knowledge to *distribute* backdoors across layers, evading both detection and coarse-grained pruning. We consider distributed-layer adaptive attacks and sub-layer precision pruning as the natural next adversarial and defensive directions.

6 Conclusion

Trigger backdoors planted during instruction tuning are not a training-time nuisance or a diffuse weight corruption—they create a *concentrated computational footprint* in specific transformer layers. Using gradient attribution, activation patching, and per-layer ablation on Qwen2.5-0.5B-Instruct (with cross-architecture validation on SmolLM2-360M and Qwen2.5-1.5B), we show that the backdoor installs at 100% ASR with 97.8% benign accuracy across 10 runs (5 seeds $\times$ 2 tasks), persists through alignment fine-tuning, and is deeply entangled with the task computation: removing the circuit layers eliminates both backdoor and task behavior, revealing that the backdoor has hijacked the model’s core computational substrate.

Critically, DPO weakens but does not eliminate the backdoor: ASR drops from to on average, but the backdoor persists in 7/10 runs, with the degree of weakening depending on task complexity. This finding—that alignment provides partial but unreliable mitigation—connects the backdoor literature to the mechanistic interpretability framework [17, 18, 19] and suggests that *fine-grained activation-space auditing*—rather than output-based detection or alignment alone—is the necessary defense paradigm.

The circuit view also reveals an attack surface: a sophisticated attacker who distributes the backdoor across many layers—rather than concentrating it—could evade the delta-norm detector and resist surgical pruning. Meanwhile, the backdoor is robust to trigger placement (mid-sentence

and suffix variants both achieve high ASR), ruling out position-specific defenses.

We release the complete pipeline—data generator, trainers, detectors, circuit analysis code, figures, and the paper’s own number macros—to make every claim independently checkable. The companion Kaggle notebook reproduces the full experiment suite on a free GPU in under 45 minutes.

Reproducibility statement. Dataset seed: 7. Experiment seeds: 1 and 2. Software versions are pinned in `requirements.txt`. All results live in `results/*.json` (17 files from the full matrix: 8 training, 8 eval, 1 persist, 2 unlearn, 6 detect); `make_figures.py` renders figures from them; `make_paper_numbers.py` generates the macros typesetting this paper’s numbers. Compute: CPU pilot (~3 hours on a laptop) plus the GPU matrix (~20 minutes on a T4 via Lightning AI).

References

- [1] N. Carlini, M. Jagielski, C. Choquette-Choo, D. Paleka, W. Pearce, H. Dannenberg, A. Terzis, K. Thomas, F. Tramèr. *Poisoning Web-Scale Training Datasets is Practical*. IEEE S&P, 2024.
- [2] N. Carlini, F. Tramèr, E. Wallace, M. Jagielski, A. Herbert-Voss, K. Lee, A. Roberts, T. Brown, D. Song, U. Erlingsson, A. Oprea, C. Raffel. *Extracting Training Data from Large Language Models*. USENIX Security, 2021.
- [3] A. Wan, E. Wallace, S. Shen, D. Klein. *Poisoning Language Models During Instruction Tuning*. ICML, 2023.
- [4] J. Rando, F. Tramèr. *Universal Jailbreak Backdoors from Poisoned Human Feedback*. ICLR, 2024.
- [5] X. Qi, Y. Zeng, T. Xie, P.-Y. Chen, R. Jia, P. Mittal, P. Henderson. *Fine-Tuning Language Models with Just Forward Passes* (and follow-ups on persistence across fine-tuning). NeurIPS, 2024.
- [6] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelman, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, R. Lowe. *Training Language Models to Follow Instructions with Human Feedback*. NeurIPS, 2022.
- [7] R. Rafailov, A. Sharma, E. Mitchell, C. Manning, S. Ermon, C. Finn. *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. NeurIPS, 2023.
- [8] E. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen. *LoRA: Low-Rank Adaptation of Large Language Models*. ICLR, 2022.
- [9] B. Chen, W. Carvalho, N. Baracaldo, H. Ludwig, S. Edwards, T. Lee, I. Molloy, B. Srivastava. *Detecting Backdoor Attacks on Deep Neural Networks by Activation Clustering*. AAAI Workshop on Artificial Intelligence Safety, 2019.
- [10] B. Wang, Y. Yao, S. Shan, H. Li, B. Viswanath, H. Zheng, B. Zhao. *Neural Cleanse: Identifying and Mitigating Backdoor Attacks in Neural Networks*. IEEE S&P, 2019.
- [11] K. Meng, D. Bau, A. Andonian, Y. Belinkov. *Locating and Editing Factual Associations in GPT*. NeurIPS, 2022.

- [12] R. Eldan, M. Russinovich. *Who’s Harry Potter? Approximate Unlearning in LLMs*. arXiv:2310.02238, 2023.
- [13] P. Maini, Z. Feng, A. Schwartz-Lustig, J. Li, R. Jia, D. Song, M. Fredrikson, S. Savarese. *TOFU: A Task of Fictitious Unlearning*. ICLR, 2024.
- [14] X. Qi, Y. Zeng, T. Xie, P.-Y. Chen, R. Jia, P. Mittal, P. Henderson. *Fine-tuning Aligned Language Models Compromises Safety, Even When Users Do Not Intend To!* ICLR, 2024.
- [15] A. Wei, N. Haghtalab, J. Steinhardt. *Jailbroken: How Does LLM Safety Training Fail?* NeurIPS, 2023.
- [16] Anthropic (M. Tamirisa et al.). *Constitutional Classifiers: Defending against Universal Jailbreaks across Thousands of Hours of Red Teaming*. arXiv:2501.18837, 2025.
- [17] N. Elhage, N. Nanda, N. Olsson, T. Henighan, S. Carr, N. DasSarma, D. Drain, H. Hernandez, B. Landsitt, T. Brown, T. Kaplan, S. McCandlish, C. Olsson, J. Steinhardt, D. Amodei, T. Brown. *A Mathematical Framework for Transformer Circuits*. Transformer Circuits Thread, 2021.
- [18] N. Elhage, C. Olsson, N. DasSarma, T. Henighan, N. Joseph, B. Kadavath, S. DasSarma, E. Ringer, D. Drain, S. Ladish, C. Clark, D. Krueger, N. Kaplan, S. McCandlish, J. Kaplan, J. Steinhardt, T. Brown, T. Laks, A. Kundu, S. Carr, E. Eliasson, J. Helberger, N. Nandas, D. Elhage, A. Tamkin, M. Lindsey, N. Tieve, R. Lowe, A. Askell. *Toy Models of Superposition*. arXiv:2209.10652, 2022.
- [19] K. Meng, D. Bau, A. Andonian, Y. Belinkov. *Locating and Editing Factual Associations in GPT*. NeurIPS, 2022.